National University of computer and emerging sciences

Artificial Intelligence Lab (AI-2002)

Semester: Spring 2026
Section: BCS-6A & BCS-6F
Course Instructor: Mr. Abdullah Shaikh

LAB # 06 -TASKS

Introduction to Genetic Algorithm and its Implementation to Solve Problems

Submission Instructions:

- Perform all tasks. Late submissions will not be considered.
- Submit a .ipynb file with the filename formatted as:
  Lab 06 - Your Name - Roll Number
- Upload the file on GCR.

TASK # 01

The National Art Museum has 7 exhibition halls arranged in a 7×7 grid. The museum wants to install 7 security cameras, with the following constraints:

1. One camera per row – no two cameras should be in the same row.
2. One camera per column – no two cameras should be in the same column.
3. No diagonal conflicts – no two cameras should "see" each other along diagonals (like laser sight interference).
4. Maximum coverage – all halls must be monitored with no blind spots caused by conflicting camera placements.

The museum wants to automate the camera placement process using a Genetic Algorithm (GA).

You are required to perform the following tasks:

1. **Chromosome Representation:**
   - Represent a camera placement as a chromosome.
2. **Fitness Function:**
   - Evaluate a chromosome by counting the number of non-conflicting pairs of cameras.
3. **Selection Technique:**
   - Implement a selection method for picking parent chromosomes.
   - Example: **Roulette Wheel Selection** or **Tournament Selection**.
4. **Crossover Technique:**
   - Implement a crossover method suitable for permutation-based chromosomes.
   - Example: **Order Crossover (OX)**.
5. **Mutation Technique:**
   - Implement a mutation method to maintain diversity.
   - Example: **Swap Mutation** – randomly swap two camera positions.
6. **Termination Condition:**
   - Stop the GA when a chromosome achieves **maximum fitness (28)** or after a fixed number of generations.
7. **Output:**
   - Print the **best camera placement** found and its **fitness value**.
   - Example: Camera Placement: [1, 3, 5, 0, 6, 4, 2], Fitness: 28

TASK # 02

You are given a population of candidate camera placements for a 7-Camera Museum GA problem. Each candidate has a fitness value as follows:

Chromosome	Fitness
C1	12
C2	20
C3	15
C4	8
C5	25

Implement the following selection techniques to pick a parent chromosome for the next generation:

- Roulette Wheel Selection
- Tournament Selection
- Rank Selection

Instructions:

- Write Python functions for each selection technique.
- Provide 2–3 lines description of each technique.
- Test your functions on the fitness table above.
- Print the selected chromosome for each method.
- Run multiple times to observe how selection varies.

```
import random

# Population and fitness table
chromosomes = ['C1','C2','C3','C4','C5']
fitnesses = [12, 20, 15, 8, 25]

# 1. Roulette Wheel Selection
def roulette_wheel_selection(chromosomes, fitnesses):
    # TODO: implement roulette wheel selection
    pass

# 2. Tournament Selection
def tournament_selection(chromosomes, fitnesses, k=3):
    # TODO: implement tournament selection
    pass

# 3. Rank Selection
def rank_selection(chromosomes, fitnesses):
    # TODO: implement rank selection
    pass

# Test each selection technique
print("Roulette Wheel Selection:", roulette_wheel_selection(chromosomes, fitnesses))
print("Tournament Selection:", tournament_selection(chromosomes, fitnesses))
print("Rank Selection:", rank_selection(chromosomes, fitnesses))
```

TASK # 03

You are given two parent chromosomes for a 7-Camera Museum GA problem:

Parent	Chromosome (list of columns for 7 rows)
P1	[1, 3, 5, 0, 6, 4, 2]
P2	[2, 5, 1, 6, 0, 3, 4]

Implement the following crossover techniques to generate offspring:

```
import random

# Parent chromosomes
parent1 = [1, 3, 5, 0, 6, 4, 2]
parent2 = [2, 5, 1, 6, 0, 3, 4]

# 1. One-Point Crossover
def one_point_crossover(p1, p2):
    # TODO: implement one-point crossover
    pass

# 2. Two-Point Crossover
def two_point_crossover(p1, p2):
    # TODO: implement two-point crossover
    pass

# 3. Order Crossover (OX)
def order_crossover(p1, p2):
    # TODO: implement order crossover
    pass

# Test each crossover technique
print("One-Point Crossover:", one_point_crossover(parent1, parent2))
print("Two-Point Crossover:", two_point_crossover(parent1, parent2))
print("Order Crossover:", order_crossover(parent1, parent2))
```

TASK # 04

You are given a chromosome representing camera placement in the 7×7 Museum grid:

Chromosome	Genes (columns for 7 rows)
C1	[1, 3, 5, 0, 6, 4, 2]

Implement the following mutation techniques to introduce diversity:

```
import random

# Original chromosome
chromosome = [1, 3, 5, 0, 6, 4, 2]

# 1. Swap Mutation
def swap_mutation(chromo):
    # TODO: select two positions randomly and swap
    pass

# 2. Scramble Mutation
def scramble_mutation(chromo):
    # TODO: select a random subset and shuffle
    pass

# 3. Inversion Mutation
def inversion_mutation(chromo):
    # TODO: select a segment and reverse it
    pass

# Test each mutation technique
print("Swap Mutation:", swap_mutation(chromosome.copy()))
print("Scramble Mutation:", scramble_mutation(chromosome.copy()))
print("Inversion Mutation:", inversion_mutation(chromosome.copy()))
```

THE END!